DSC 102: Systems for Scalable Analytics

Programming Assignment 3: Machine Learning Competition Challenge 1

Released: 26 October 2026, Due: 13 November 2026 (11:59 PM Pacific)

VERY IMPORTANT: There are no late days for this assignment, so plan your work accordingly. You may submit to the AI Coding Gym leaderboard as many times as you like and only your best score is recorded, so submit early and often. Do not download the original Kaggle data or look up existing solutions to this competition (see Section 6). GPU resources on DataHub are limited, and you may have to wait in a queue before you get a GPU (see Section 5). Unless you have your own compute, start early.

1 Introduction

In this assignment you will work on a real machine learning competition from AI Coding Gym, **Ventilator Pressure Prediction**, where frontier AI agents fall short. Your goal is to outperform them by applying your own judgment, running careful experiments, and using AI assistants as collaborative tools. Your leaderboard score matters, but understanding matters more. A perceptive analysis of what worked, what did not, and why is the core of this assignment.

Challenge page:
https://aicodinggym.com/challenges/mle/ventilator-pressure-prediction

2 Problem and Dataset

You will predict the airway pressure in the respiratory circuit during each breath, given the time series of control inputs and the attributes of the lung.

The data was produced by a modified open-source ventilator connected to an artificial bellows test lung via a respiratory circuit. Each time series is a breath of about 3 seconds, and each row is one time step of a breath. A row gives the two control inputs, the resulting airway pressure, and the relevant attributes of the lung. The first control input, `u_in`, is the percentage (0 to 100) that the inspiratory solenoid valve is open to let air into the lung. The second, `u_out`, is binary: whether the expiratory valve is open (1) or closed (0) to let air out.

File	Contents
`train.csv`	The training set (with `pressure`)
`test.csv`	The test set (without `pressure`)
`sample_submission.csv`	A sample submission file in the correct format

Table 1: Files provided by `aicodinggym mle download`

Column	Description
`id`	Globally unique time step identifier across an entire file
`breath_id`	Globally unique identifier of a breath
`R`	Lung attribute indicating how restricted the airway is ($cmH_2O/L/s$): the change in pressure per change in flow (air volume per time). Imagine blowing up a balloon through a straw: `R` is set by the straw's diameter, and higher `R` is harder to blow.
`C`	Lung attribute indicating how compliant the lung is (mL/cmH_2O): the change in volume per change in pressure. In the same balloon example, `C` is set by the thickness of the balloon's latex; higher `C` means thinner latex, easier to blow.
`time_step`	The actual time stamp
`u_in`	Control input for the inspiratory solenoid valve, from 0 to 100
`u_out`	Control input for the expiratory solenoid valve, either 0 or 1
`pressure`	Airway pressure measured in the respiratory circuit (cmH_2O). This is the target; it is not in `test.csv`.

Table 2: Schema of `train.csv` and `test.csv`

3 Evaluation and Leaderboard

Submissions are scored by the mean absolute error (MAE) between the predicted and actual pressures during the inspiratory phase of each breath. The expiratory phase is not scored. Lower is better.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|X_i - Y_i|$$

Here X is the vector of predicted pressures and Y is the vector of actual pressures across all breaths in the test set, taken over the n scored (inspiratory) time steps.

Submission format. For each `id` in the test set, you must predict a value for the `pressure` variable. The file must contain a header and follow the format below (see also `sample_submission.csv`):

```
id,pressure
1,20
2,23
3,24
```

Leaderboard. Your best submission is ranked on the AI Coding Gym leaderboard for this challenge. Table 3 gives reference points: the three AI agents from the challenge page's AI Benchmark panel, and the median and medal lines of the original competition leaderboard.

Reference point	MAE
Gold medal line	0.1119
Silver medal line	0.1338
Bronze medal line (top 10%)	0.1364
AI agent: MLEvolve	0.14237
AI agent: Disarray	0.15171
Leaderboard median	0.1638
AI agent: AIBuildAI	1.0759

Table 3: Leaderboard reference points

4 What You Will Do

Work through the following loop, repeating steps 2 and 3 as often as your time allows.

1. **Inspect the AI agents' work (recommended).** The AI Benchmark panel on the challenge page has a "source" link to the agents' runs. Study their decisions: how they processed the data, which features and model they chose, how they validated, and what they skipped or got wrong. Note that one agent scored roughly seven times worse than the others; working out why is a good exercise.

2. **Build your own solution with AI as a tool.** Start with a simple, valid submission, then improve it. Use any AI coding assistant you like, but apply your own judgment and override its decisions where you see weaknesses. Change one thing at a time and keep a log of your experiments and their validation scores; you will need it for the report. Also note where your AI assistant made mistakes or weak suggestions and how you caught them; diagnosing your own AI is a key part of the report.

3. **Submit and compare.** Submit your predictions on AI Coding Gym. Your score appears on the live leaderboard next to the AI agents and your classmates.

4. **Share work in progress in class.** Two or three students will present their approaches for 10 minutes each, chosen for the variety of their methods, not for top scores. Dates will be announced.

5. **Write the report** described in Section 6.

5 Compute and Platform Setup

You will train on a GPU container on UCSD DataHub and use the AI Coding Gym command-line tool to download the data and submit predictions. You may also work on any other machine you have access to. Full DataHub documentation: Launching Containers From the Command Line.

Start a GPU container.

1. If you are off campus, connect to the UCSD VPN.
2. Log in to the launch server with your UCSD credentials:

   ```
   ssh <username>@dsmlp-login.ucsd.edu
   ```

   Do not run code on this server; it only launches containers.
3. Launch the course GPU environment:

   ```
   launch-scipy-ml.sh -W DSC102_FA26_001 -g 1 -c 8 -m 16
   ```

   If all GPUs are busy, your job waits in a queue; do not press Ctrl-C, or you lose your place.
4. When the container starts, the command prints a line like

   ```
   You may access your Jupyter notebook at:
     http://dsmlp-login:<port>/user/<username>/?token=<token>
   ```

   Open the same link with the full host name in your browser:

   ```
   http://dsmlp-login.ucsd.edu:<port>/user/<username>/?token=<token>
   ```
5. In JupyterLab, open File → New → Terminal and run `nvidia-smi` to check that you have a GPU.

Download the data. In the container's terminal, run:

```
pip install aicodinggym-cli
echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
export PATH=$HOME/.local/bin:$PATH
aicodinggym configure --user-id <your-user-id>
aicodinggym mle download ventilator-pressure-prediction
cd ~/ventilator-pressure-prediction/data
unzip ventilator-pressure-prediction.zip
```

Your user ID is on your AI Coding Gym profile page. You only need to configure once; your home directory is kept between containers.

Submit predictions.

```
aicodinggym mle submit ventilator-pressure-prediction -F submission.csv
```

Your score is printed right away and also appears on the challenge page.

Limits and tips.

- The course GPU environment uses a shared pool of NVIDIA GTX 1080 Ti GPUs (11 GB). The pool is shared with other DataHub users, so you may wait in a queue before a GPU is free, and waits get longer near deadlines. Unless you have your own compute, start early.

- Each user can run at most 1 GPU, 8 CPUs and 64 GB of RAM at a time.
- A container stops after 6 hours by default. To extend it to at most 12 hours, set the variable `K8S_TIMEOUT_SECONDS` before launching, for example `K8S_TIMEOUT_SECONDS=43200` followed by your launch command.
- A container also stops if you disconnect from dsmlp-login. For long runs, add `-b` to run it in the background and reconnect later with `kubesh <pod-name>`.
- Save checkpoints regularly, so a stopped container does not lose your training.
- When you are done, free the GPU for your classmates: run `kubectl get pod` to find your pod's name, then `kubectl delete pod <pod-name>`.
- For DataHub problems, email datahub@ucsd.edu with the course ID (DSC102_FA26_001), the command you ran and a screenshot.

6 Deliverables and Rules

Submit three things by the deadline. The rules for each are listed with it.

1. **Challenge submission on AI Coding Gym.** Submit your predictions on the platform as many times as you like; your best score is recorded. Put your AI Coding Gym username on Canvas so we can find you on the leaderboard.
   - Use only the data downloaded with `aicodinggym mle download`. Do not download the original Kaggle competition data from Kaggle or any mirror, or any dataset or model built from it. The hidden test set was taken from that data, so using it leaks the answers.
2. **AI interaction history on Canvas.** Submit the complete history of prompts, AI responses, and meaningful iterations that influenced your final solution. This history is the evidence for the AI diagnosis in your report.
   - You may use any AI tools you like.
   - If an AI assistant suggests a trick specific to this competition without being asked, you may use it, but point it out in your history and in your report.
3. **Final report on Canvas.** A PDF of three to five paragraphs covering:
   - **AI inspection summary:** what your own AI assistant did, its key decisions about models, preprocessing and features, and the errors and weaknesses you found in its suggestions and reasoning. You may also compare it with the benchmark agents from Section 3.
   - **Your strategy:** what you changed and why, the prompts you used with AI assistants, and the judgment you applied that the AI did not.
   - **Results comparison:** your score compared with the AI agents, with a screenshot of your leaderboard position.
   - **Reflection:** where the AI fell short, what you did that it did not, and what concept or skill this taught you.

Existing solutions. Do not look for existing solutions to this competition. This includes Kaggle notebooks, discussion threads, winners' write-ups, GitHub repositories and blog posts about it, and asking an AI assistant what the winning teams did. Ideas you reach by analyzing the data yourself are welcome, even if they match what top teams found. Credit any idea that came from an outside source in your report.